

Open in app ↗

Medium

🔍 Search

✍ Write



S

★ Get unlimited access to the best of Medium for less than \$1/week. [Become a member](#)



Common Design Patterns In Android



Ani Mansoubi · [Follow](#)

11 min read · Feb 21, 2024



170



3



When we are talking about design patterns we are talking about reusable solutions for solving many problems in Software development.

One of the important questions that came to my mind when I encountered Design pattern concepts at the very first glance was why and how a grasp of Design pattern concepts makes you a senior developer.

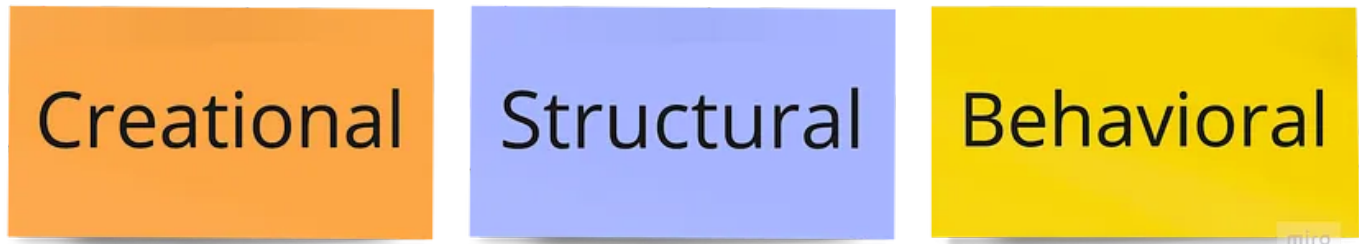
I've dug into that and found, that the story goes back to a long time ago when Christopher Alexander in the late 1970s decided to talk about successful solutions to the problems they faced. It seems they faced recurring problems that could have been solved with patterns, thus they documented many solutions to the problems.

Many developers like “Gang of Four” were inspired by these theories and they applied this concept in software development and called it the OOP Design pattern.

Let's discover what Design patterns are :

Elegant solutions for repeating problems is a design pattern that shows you how to structure your classes and how these classes should talk to each other to solve a problem. This article discusses some of the important patterns you should know as an Android developer.

If we want to categorize Design patterns they go into 3 categories:



1. Creational is all about different ways to create objects.
2. Structural patterns are about relationships between these objects.
3. Behavioral patterns are all about interaction and communication between these objects.

knowing this design pattern can help you to communicate with other developers at a more abstract level, it is the same language for transforming ideas to your teammate instead of writing a lot of codes and also building extensible and maintainable software. Furthermore, knowing design patterns helps you learn the new framework easily and quickly.

Knowing the art of Object Oriented Programming is essential for Knowing design patterns but I assume you know these concepts as an Android developer whether you are coding with Java or Kotlin.

- **Creational Patterns**

1. Singleton Pattern

You might know what Singleton is. but you do not know it is a Design Pattern. Singleton is an object is just has one instance through your entire code base (Application). In kotlin with an **object** keyword you can create a singleton as easily as below:

```
object Singelton {  
    fun doSomeTask(){  
    }  
}
```

If you want to go deeper and see what is behind the scenes and what has been implemented in Java you could take a look at the code below:

```
public final class Singelton {  
    @NotNull  
    public static final Singelton INSTANCE;  
  
    public final void doSomething(){  
  
    }  
  
    private Singelton(){  
    }  
  
    static {  
        Singelton var(0) = new Singelton();  
        INSTANCE = var(0);  
    }  
}
```

2. Factory Pattern

The Factory Pattern is a valuable tool for object creation in software development. It proves particularly useful in various scenarios. For instance, when dealing with fragments in Android development, using the Factory Pattern is highly recommended. This is because during events like configuration changes or where the Android platform needs to recreate the fragment, keeping track of all the parameters within the fragment becomes challenging without a proper factory mechanism.

```
class AddEventFragment constructor() : Fragment() {

    private lateinit var param: String

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        param = savedInstanceState?.getString(KEY) ?: arguments?.getString(KEY)
    }

    @Override
    fun showEvent() {
        //Show event detail
    }

    companion object {
        fun newInstance(param: String): AddEventFragment {
            val eventDetailFragment = AddEventFragment()
            val bundle = Bundle().apply {
                putString(KEY, param)
            }
            eventDetailFragment.arguments = bundle
            return eventDetailFragment
        }

        private const val KEY = "PARAM_KEY"
    }
}

//Usage
val fragment = AddEventFragment.newInstance("PARAM")
```

Another case would be Using a factory to decide and distinguish between similar types of objects and then create a desire for one of them like the code below:

```
enum class DialogType {  
    DIALOG_TO_EDIT,  
    DIALOG_TO_DELETE,  
    DIALOG_TO_ARCHIVE,  
}  
  
sealed class Dialog {  
    data object EditDialog : Dialog()  
    data object DeleteDialog : Dialog()  
    data object ArchiveDialog : Dialog()  
}  
  
object DialogFactory {  
    fun createDialog(dialogType: DialogType): Dialog {  
        return when (dialogType) {  
            DialogType.DIALOG_TO_EDIT -> Dialog.EditDialog  
            DialogType.DIALOG_TO_DELETE -> Dialog.DeleteDialog  
            DialogType.DIALOG_TO_ARCHIVE -> Dialog.ArchiveDialog  
        }  
    }  
}
```

3. Builder Pattern

The Builder Pattern is yet another creational design pattern that facilitates the creation of objects while allowing for optional modifications on a single object. To illustrate, imagine you're crafting a handmade pizza. With a plethora of options available, you can customize and build your pizza just the way you like it. Let's explore the Pizza class to better understand this concept.

```

class Pizza private constructor(
    val whiteSauce: Boolean = false,
    val redSauce: Boolean = false,
    val garlicSauce: Boolean = false,
    val onion: Boolean = false,
    val olive: Boolean = false,
    val salami: Boolean = false,
    val chicken: Boolean = false,
    val beef: Boolean = false,
    val mozzarellaCheese: Boolean = false,
    val goudaCheese: Boolean = false,
) {
    class Builder() {

        private var whiteSauce: Boolean = false
        private var redSauce: Boolean = false
        private var garlicSauce: Boolean = false
        private var onion: Boolean = false
        private var olive: Boolean = false
        private var salami: Boolean = false
        private var chicken: Boolean = false
        private var beef: Boolean = false
        private var mozzarellaCheese: Boolean = false
        private var goudaCheese: Boolean = false
        fun addWhiteSauce(value: Boolean) = apply { whiteSauce = value }
        fun addRedSauce(value: Boolean) = apply { redSauce = value }
        fun addGarlicSauce(value: Boolean) = apply { garlicSauce = value }
        fun addOnion(value: Boolean) = apply { onion = value }
        fun addOlive(value: Boolean) = apply { olive = value }
        fun addSalami(value: Boolean) = apply { salami = value }
        fun addChicken(value: Boolean) = apply { chicken = value }
        fun addBeef(value: Boolean) = apply { beef = value }
        fun addMozzarellaCheese(value: Boolean) = apply { mozzarellaCheese = value }
        fun addGoudaCheese(value: Boolean) = apply { goudaCheese = value }
        fun build() = Pizza(
            whiteSauce = whiteSauce,
            redSauce = redSauce,
            garlicSauce = garlicSauce,
            onion = onion,
            olive = olive,
            salami = salami,
            chicken = chicken,
            beef = beef,
            mozzarellaCheese = mozzarellaCheese,
            goudaCheese = goudaCheese
        )
    }
}

```

```
}  
}
```

```
class MainActivity : AppCompatActivity() {  
    override fun onCreate(savedInstanceState: Bundle?) {  
        super.onCreate(savedInstanceState)  
        val pizza =  
            Pizza.Builder()  
                .addBeef(true)  
                .addOlive(true)  
                .addOnion(true)  
                .addMozzarellaCheese(true)  
                .addWhiteSauce(true)  
    }  
}
```

Absolutely, in Kotlin, we can achieve similar functionality using data classes, but let's stick with the pizza analogy as it serves as a straightforward example to grasp the builder pattern concept.

4. Facade Pattern

Let's head over to the next Creational Pattern which you may not be familiar with yet because it is not so frequently used but it is still very helpful and is called the Facade Pattern. I will show you that with the Retrofit interface as an example. Facade pattern is used to hide code that you don't need to see. that's exactly what the Retrofit API interface does here below:

```
interface EventRemoteService {  
    @GET("all_event")  
    suspend fun getAllEvents(  
        @Query("page_size") pageSize: Int,  
        @Query("page_number") pageNumber: Int,
```



```
    ): List<EventModel>  
}
```

You don't need to worry about the inner Retrofit workings when handling network calls. All you need is this function to retrieve all your events from the server. We point to the term "Facade" for this pattern because it allows you to interact with the surface of things without concern about complexities.

5. Dependency Injection Pattern

This is what every single Android developer must know, illustrated by examples like dependency injection in Hilt modules which provides our remote service for every class that needs EventRemoteService, as demonstrated in the code below:

```
@Module  
@InstallIn(SingletonComponent::class)  
class EventModule {  
    @Provides  
    @Singleton  
    fun provideEventRemoteService(  
        @AuthorizedRetrofitQualifier retrofit: Retrofit  
    ): EventRemoteService {  
        return retrofit.create(EventRemoteService::class.java)  
    }  
}
```

Understanding this pattern is vital for developers as it enables the seamless provision of dependencies to your classes. Whether through constructor injection or field injection. Frameworks like Dagger or, Hilt which is based

on Dagger, simplify the process, making it more efficient and painless for us. Also using DI makes your app more testable and makes the test process more convenient.

- **Structural Patterns**

1. Adapter Pattern

The Adapter Pattern is something every Android developer, whether just starting or seasoned, is familiar with. When it comes to implementing RecyclerViews, utilizing an adapter is essential for efficiently setting the list of data to your RecyclerView.

it's my adapter in case you need a quick reminder:

```
@FragmentScoped
class EventsAdapter @Inject constructor(
    @ApplicationContext private val context: Context,
) : RecyclerView.Adapter<EventsAdapter.ViewHolder>() {

    private var eventItemList: MutableList<Event> = mutableListOf()
    private val li = LayoutInflater.from(context)

    fun updateEvents(newEventList: List<Event>) {
        val distinctNewEvents = newEventList.distinctBy { it.eventId }
        val diffCallback = EventsDiffCallback(eventItemList, distinctNewEvents)
        val diffResult = DiffUtil.calculateDiff(diffCallback)

        eventItemList.clear()
        eventItemList.addAll(distinctNewEvents)

        diffResult.dispatchUpdatesTo(this)
    }

    override fun onCreateViewHolder(
        parent: ViewGroup, viewType: Int
    ): ViewHolder {
        val itemBinding = ItemEventBinding.inflate(li, parent, false)
        return ViewHolder(
```

```
        itemBinding = itemBinding,
    )
}

override fun getItemCount(): Int = eventItemList.size

override fun onBindViewHolder(holder: ViewHolder, position: Int) {
    return holder.bind(eventItemList[position])
}

inner class ViewHolder(
    private val itemBinding: ItemEventBinding,
) : RecyclerView.ViewHolder(itemBinding.root) {

    fun bind(event: Event) {
        itemBinding.imgEdit.isVisible = false
        itemBinding.tvEventTitle.text = event.eventName
        itemBinding.tvEventDescription.text = event.description
        itemBinding.tvEventHost.text = event.eventHostEmail
        val resId = if (event.liked) R.drawable.ic_liked else R.drawable.ic_
        itemBinding.imgFavorite.setImageResource(resId)
    }
}
}
```

2. Decorator Pattern

In the Decorator pattern, you have a base component interface or class and then you can easily modify the functionality of an object at runtime, and at the same time, other instances of the same class will not be affected by this, so the individual object gets the modified behavior.

In our example, we have a `Book` class that encapsulates a title and a description. To enhance the behavior of the `Book` objects dynamically, we utilize the Decorator Pattern.

```
open class Book(  
    var title: String,  
    var description: String  
)  
  
data class BookWithAuthor(var bookTitleWithAuthor: Book, var author: String) :  
    Book(  
        title = bookTitleWithAuthor.title + "_" + author,  
        description = bookTitleWithAuthor.description  
    )  
  
data class BookWithSource(var bookTitleWithAuthor: Book, var source: String) :  
    Book(  
        title = bookTitleWithAuthor.title,  
        description = bookTitleWithAuthor.description + "_" + source  
    )
```

We extend the functionality of the `Book` class by creating specialized decorators. For instance, we have a `BookWithAuthor` decorator, which inherits from, enriching the title by appending the author's name. Similarly, the `BookWithSource` decorator augments the description by including the source. By using decorators, we can alter the behavior of individual `Book` objects without modifying the core functionality of the `Book` class. This promotes flexibility and maintainability in our codebase, allowing for modular and reusable enhancements.

- **Behavioral Pattern**

1. Observer Pattern

The Observer Pattern stands out as a fundamental pattern in Android development. Picture a scenario where you must track changes in your `ViewModel`'s state and reflect these changes in your corresponding view.

Your ViewModel encapsulates each state change within a channel, while your view subscribes to this channel, consistently monitoring for updates and adjusting the view accordingly. This concept is beyond the viewModel but it is the more sensible example for Android developers. We can implement this pattern like below:

```
interface Observer {
    fun update(data: String)
}

interface Observable {
    fun register(observer: Observer)
    fun deRegister(observer: Observer)
    fun notifyChanges(data: String)
}

class Chanel : Observable {

    private val observers = mutableListOf<Observer>()
    override fun register(observer: Observer) {
        observers.add(observer)
    }

    override fun deRegister(observer: Observer) {
        observers.remove(observer)
    }

    override fun notifyChanges(data: String) {
        observers.forEach {
            it.update(data)
        }
    }

    fun setData(data: String) {
        notifyChanges(data)
    }
}

class DataDisplay() : Observer {
    override fun update(data: String) {
        println("new data was published!!")
    }
}
```

```
}  
// Register to the chanel and get updated data  
val chanel = Chanel()  
val dataDisplay = DataDisplay()  
chanel.register(dataDisplay)  
chanel.setData("new data here")
```

2. Command Pattern

The Command Pattern is a design pattern that can be implemented to encapsulate a request as an object, allowing the client to parameterize and execute the request at a later point in time. It proves particularly beneficial when developing applications where the client needs to queue, log, or undo operations. You can implement this pattern like this :

```
interface Log {  
    fun execute()  
}  
  
class PrintLog(private val message: String) : Log {  
    override fun execute() {  
        println(message)  
    }  
}  
  
class LogsInvoker() {  
    private val logs = mutableListOf<Log>()  
    fun addLog(log: Log) {  
        logs.add(log)  
    }  
  
    fun executeLogs() {  
        logs.forEach {  
            it.execute()  
        }  
        logs.clear()  
    }  
}
```

```
fun run() {  
    val logsInvoker = LogsInvoker()  
    logsInvoker.addLog(PrintLog("Server is response very soon ...!"))  
    logsInvoker.addLog(PrintLog("Auth token is gfhgjkghkjgyjdhsjg."))  
    logsInvoker.addLog(PrintLog("the Add Event API call response code is 200."))  
    logsInvoker.executeLogs()  
}
```

In our example, we have logs to print but we want to print them sometime in the future. You can add logs in different classes to your application and just call the execute fun at an appropriate time. In summary command pattern focuses on encapsulating requests (in our case logs) as an object and decoupling requests in a time we need (executing the logs).

3. Chain Pattern

The Chain of Responsibility design pattern enables an object to delegate a request to a chain of potential handlers without specifying the exact handler. Let's illustrate this with a scenario: imagine a customer calls customer service with a request. If the customer service representative cannot address the request or requires additional details, they pass it on to their supervisor. The supervisor, if unable to handle the request or need further clarification, forward it to the manager. This process continues until the request is addressed by someone within the organization. ▶

In essence, the Chain of Responsibility pattern functions as a relay mechanism, where each handler in the chain evaluates the request and either handles it or passes it along to the next handler until it reaches a suitable resolution. This approach is developing the flexibility and scalability in managing various types of requests. Let's dive into implementation:

```
data class Request(val details: String)

// Define the abstract Handler class
abstract class Handler(private val successor: Handler? = null) {
    abstract fun handleRequest(request: Request)

    // Passes the request to the next handler in the chain if there is one
    fun passToNext(request: Request) {
        successor?.handleRequest(request)
    }
}

// Define concrete handlers
class CustomerService(private val successor: Handler? = null) : Handler(successor) {
    override fun handleRequest(request: Request) {
        if (request.details == "Complaint") {
            println("Customer Service handling complaint.")
        } else {
            passToNext(request)
        }
    }
}

class Supervisor(private val successor: Handler? = null) : Handler(successor) {
    override fun handleRequest(request: Request) {
        if (request.details == "Escalate to Supervisor") {
            println("Supervisor handling escalation.")
        } else {
            passToNext(request)
        }
    }
}

class Manager(private val successor: Handler? = null) : Handler(successor) {
    override fun handleRequest(request: Request) {
        if (request.details == "Escalate to Manager") {
            println("Manager handling escalation.")
        } else {
            println("Request cannot be handled by anyone in the organization.")
        }
    }
}

// Usage
fun main() {
    // Create the chain of responsibility
    val manager = Manager()
    val supervisor = Supervisor(manager)
```



```
val customerService = CustomerService(supervisor)

// Create a request
val request = Request("Complaint")

// Pass the request through the chain
customerService.handleRequest(request)
}
```

4. Strategy Pattern

The Strategy Design Pattern is a pattern that allows a client to choose from a family of algorithms at runtime. It encapsulates each algorithm and makes them interchangeable. Let's consider a scenario of sorting algorithms where different algorithms (Bubble Sort, Quick Sort, Merge Sort, and, etc) can be used interchangeably based on specific requirements.

```
interface SortStrategy {
    fun sort(dataset: MutableList<Int>)
}

//Bubble Sort
class BubbleSortStrategy : SortStrategy {
    override fun sort(dataset: MutableList<Int>) {
        println("Sorting using Bubble Sort")
        // Implementation of Bubble Sort
    }
}

// Quick Sort
class QuickSortStrategy : SortStrategy {
    override fun sort(dataset: MutableList<Int>) {
        println("Sorting using Quick Sort")
        // Implementation of Quick Sort
    }
}

class Sorter(private var strategy: SortStrategy) {
```

```
fun setStrategy(strategy: SortStrategy) {
    this.strategy = strategy
}

fun sort(dataset: MutableList<Int>) {
    strategy.sort(dataset)
}

}

// Client code
fun main() {
    val dataset = mutableListOf(5, 2, 7, 1, 9)

    // Using Bubble Sort
    val sorter = Sorter(BubbleSortStrategy())
    sorter.sort(dataset.toMutableList())

    // Using Quick Sort
    sorter.setStrategy(QuickSortStrategy())
    sorter.sort(dataset.toMutableList())
}
```

[Design Patterns](#)[Android Design Patterns](#)[Creational Design Pattern](#)[Structural Design Pattern](#)[Behavioral Design Pattern](#)**Written by Ani Mansoubi**

50 Followers · 67 Following

Senior Android Developer

[Follow](#)

Responses (3)



What are your thoughts?

Respond



mohammad moradyar

10 months ago



Awesome, it was a very good article about the common design patterns in Android development, Thank you so much.



1



1 reply

Reply



Erfan Moteva3li

10 months ago



Thanks for sharing this useful information about Design Patterns.



1 reply

Reply



Stancho Stanchev

10 months ago



We all Android developers needed something like that. Thanks for sharing !



1 reply

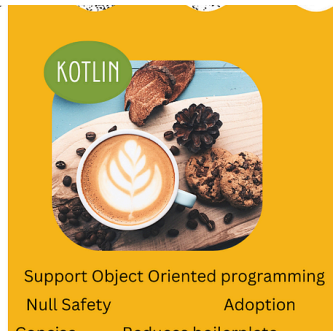
Reply

More from Ani Mansoubi



Support Object Oriented programming
Null Pointer exception

More Kotlin Java



Support Object Oriented programming
Null Safety Adoption

Creating Kotlin Kotlin



Ani Mansoubi

Java vs. Kotlin

I've accumulated over 5 years of experience in Android development using Kotlin. While...

Mar 11

👏 43

💬 1



Ani Mansoubi

Why Asynchronous Programming?

Whenever you talk about Asynchronous programming, UI (user interface) play a key...

Nov 15, 2023

👏 64

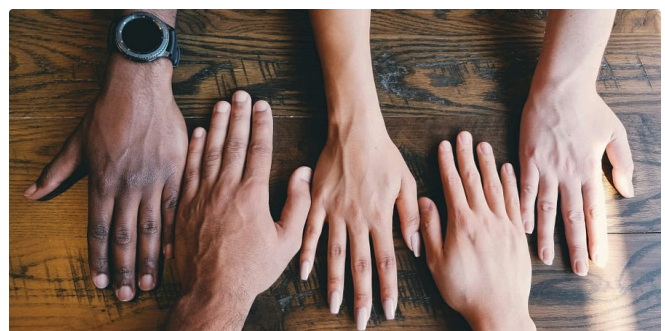


See all from Ani Mansoubi

Recommended from Medium



In Nerd For Tech by Niles B



Aman Shandilya

Retrofit: Simplify API Calls in Android

 Networking in Android Apps without Retrofit

★ Sep 19 🖱 51 💬 1



Android: Use ComposeView with RecyclerView

RecyclerView is a powerful and flexible UI component in Android development which i...

★ Aug 9 🖱 18



Lists



Staff picks

789 stories · 1507 saves



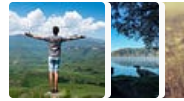
Stories to Help You Level-Up at Work

19 stories · 892 saves



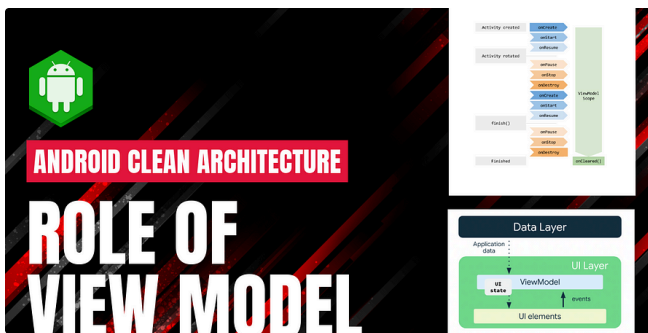
Self-Improvement 101

20 stories · 3144 saves



Productivity 101

20 stories · 2655 saves



 Dawinder Singh Gill

Android Interview Questions: 46 | The Role of ViewModel in Clean...

Why ViewModels are Essential for Lifecycle Management?

★ Sep 24 🖱 402



 In WhiteCodeL by Bhawani Shankar

Flutter Interview Questions (2024) PART—4

Welcome to the fourth chapter of our Flutter Interview Questions series for 2024! 🚀 As ...

★ Aug 10 🖱 66





 Michael Angelo Reyes

Managing UI State in Android

When developing Android apps, managing and observing UI state efficiently is crucial fo...

★ Aug 19



 Shahinoor Shahin

SOLID Principles in Android Application Development with...

SOLID is an acronym that represents five design principles introduced by Robert C....

★ Nov 24 🖱 65



See more recommendations